

Pichulman Miguel Angel

Trabajo Práctico I: "Calidad de Diseño y Construcción"

Objetivos

- Diferenciar calidad interna y calidad externa.
- Comprender el concepto de deuda técnica.
- Identificar code smells frecuentes.
- Analizar el impacto del costo del cambio.
- Aplicar mejoras básicas de mantenibilidad.

Contexto

El Hospital Central desarrolló rápidamente un sistema de gestión de turnos médicos durante la pandemia. El sistema actualmente funciona correctamente para los usuarios, pero el código fuente presenta múltiples problemas internos que dificultan su mantenimiento y evolución.

El objetivo del trabajo práctico será analizar el código existente, detectar problemas de diseño y aplicar mejoras sin modificar el comportamiento externo del sistema.

Actividades

Parte A — Análisis de Calidad

Identificar al menos 5 code smells.

- estructuras if anidadadas
- muchas estructuras if-else
- strings hardcodeados
- metodo procesar con demasiadas responsabilidades
- compracion explicita con boolean

Identificar al menos 3 problemas de mantenibilidad.

- toda la logica se encuentra dentro de un solo metodo
- las reglas del negocio dependen de valores literales (ej. "OSDE", "SWISS")
- no hay una validacion de datos como por ejemplo edad>0

Pichulman Miguel Angel

Explicar al menos 2 ejemplos de deuda técnica.

- la utilización de enum para las obras sociales
- la utilización de un objeto paciente

Describir cómo estos problemas afectan el costo del cambio.

El costo del cambio aumenta significativamente. Cualquier nueva funcionalidad requiera leer, entender y modificar un bloque de código monolítico. Modificar la forma en que se almacenan los datos de un paciente y los turnos implica el riesgo de romper la lógica de validación porque está todo mezclado.

Parte B — Calidad Interna vs Calidad Externa

Responder:

¿Qué aspectos representan calidad externa?

La calidad externa es lo que percibe el usuario final. Ejemplos: que el paciente se registre correctamente, que se muestren correctamente los turnos.

¿Qué aspectos representan calidad interna?

La calidad interna se manifiesta como los atributos técnicos visibles para el equipo de desarrollo. Ejemplo: nombres claros y distintivos, ausencia de duplicación, responsabilidades separadas.

¿Puede un sistema funcionar correctamente y aun así tener mala calidad?

Si. Un sistema puede hacer exactamente lo que el usuario necesita de manera rápida pero estar construido con un código desordenado por detrás. Aunque funcione hoy, será un dolor de cabeza modificarlo mañana.

¿Qué riesgos genera mantener deuda técnica a largo plazo?

Aumenta exponencialmente el tiempo y costo de mantenimiento. Reduce la velocidad de desarrollo (velocity) porque el equipo pasa más tiempo arreglando fallos que creando valor. En casos extremos, la deuda es tan alta que el sistema se vuelve inmantenible y se debe reescribir desde cero.

Parte C — Refactor Inicial

Realizar mejoras al código manteniendo el mismo comportamiento funcional.

Aplicar al menos las siguientes mejoras:

Mejorar nombres de variables y métodos.

Eliminar duplicación de código.

Reducir complejidad condicional.

Pichulman Miguel Angel

Separar responsabilidades.

Mejorar legibilidad general.

Importante: No debe modificarse el comportamiento observable del sistema. El refactor debe conservar la funcionalidad original.

```
import java.util.ArrayList;
import java.util.List;

public class TurnoManager {

    private static final String OSDE = "OSDE";
    private static final String SWISS = "SWISS";
    private static final String PUBLICA = "PUBLICA";
    private static final String PREMIUM = "Paciente premium";
    private static final String PUBLICO = "Paciente publico";
    private final List<String> turnos = new ArrayList<>();

    public void procesar(
        String nombrePaciente,
        int edad,
        String obraSocial,
        boolean urgente) {
        if (!datosValidos(nombrePaciente, edad)) {
            return;
        }
        mostrarTipoPaciente(obraSocial);
        String turno = construirTurno(
            nombrePaciente,
            edad,
            obraSocial,
            urgente);
        turnos.add(turno);
        System.out.println("Turno agregado");
    }

    private boolean datosValidos(String nombre, int edad) {
        return nombre != null &&
```

Pichulman Miguel Angel

```
!nombre.isEmpty() &&
    edad > 0;
}

private void mostrarTipoPaciente(String obraSocial) {
    if (OSDE.equals(obraSocial) || SWISS.equals(obraSocial)) {
        System.out.println(PREMIUM);
    } else if (PUBLICA.equals(obraSocial)) {
        System.out.println(PUBLICO);
    }
}

private String construirTurno(
    String nombre,
    int edad,
    String obraSocial,
    boolean urgente) {
    String turno = nombre + "-" + edad + "-" + obraSocial;
    if (urgente) {
        turno += "-URGENTE";
    }
    return turno;
}

public void mostrar() {
    for (String turno : turnos) {
        System.out.println(turno);
    }
}
}
```

Pichulman Miguel Angel

Mejoras aplicadas

Se mejoraron algunos nombres y se utilizaron constantes para evitar strings hardcodeados.

Se elimino la comparación redundante urgente == true.

Se redujo la complejidad condicional mediante retornos tempranos.

Se separaron responsabilidades en metodos especificos (datosValidos, mostrarTipoPaciente y construirTurno).

Se mejoro la legibilidad utilizando un bucle for-each en mostrar().

Se mantuvo el mismo comportamiento observable del sistema: las validaciones, los mensajes impresos y la generacion de turnos siguen produciendo los mismos resultados que el codigo original.